DS5003A – Data Engineering

Assignment 5: Graph Traversal

August 28, 2026

Duration: 2:00 PM – 4:45 PM
Total Questions: 7

Instructions:

- Difficulty increases progressively from Q1 to Q7.
- Follow all constraints specified in each question strictly. Use of prohibited built-in functions will result in loss of marks.
- Write clean, well-commented Python code.
- Please call the TA once you have finished the assignment for marking and the mini-viva.

Question 1: Graph Representation and Breadth-First Search

Given a set of edges representing connections between different systems, write a Python program to construct the graph using an **adjacency list** and perform **Breadth-First Search (BFS)** from a given starting node.

Input

```
Edges:
A-B
A-C
B-D
C-E
D-F

Starting node: A
```

Expected BFS traversal

```
A -> B -> C -> D -> E -> F
```

Constraints

- Represent the graph using a Python dictionary.
- Implement BFS manually.
- Use a queue and a `visited` set.
- Do not use any external graph library.

Question 2: Depth-First Search Using Stack

A directed graph represents dependencies between different services. Write a Python program to perform **Depth-First Search (DFS)** from a given starting node.

The DFS must be implemented using an **explicit stack**.

Input

```
Graph:
A -> B, C
B -> D
C -> E
D -> F
E -> F

Starting node: A
```

Display the DFS traversal order.

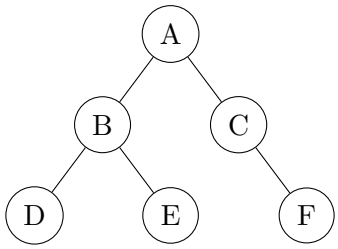
Constraints

- Use a stack.
- Use a `visited` set.
- Do not use recursion.
- Do not use external graph libraries.

Question 3: Find Nodes at a Given Distance

Given an unweighted graph and a starting node, find all nodes that are exactly **K edges away** from the starting node.

Input




```
Starting node: A
K = 2
```

Expected output

```
Nodes at distance 2:
D E F
```

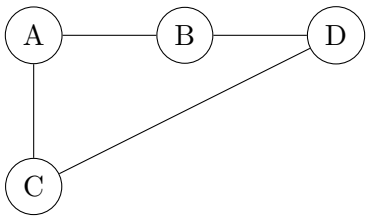
Constraints

- Use BFS.
- Nodes must be exactly `K` edges away.
- Do not use any shortest-path or graph library.

Question 4: Find the Shortest Route

A network of locations is represented using an unweighted graph. Write a Python program to find the **shortest path between two given nodes**.

Input




```
Source: A
Destination: D
```

Expected output

```
Shortest path: A -> B -> D
Number of edges: 2
```

If no path exists, display:

```
No path exists
```

Constraints

- Use BFS.
- Display the actual shortest path.
- Do not use any shortest-path library.

Question 5: Find Connected Components

Given an **undirected graph**, determine the number of disconnected groups of nodes and display the nodes belonging to each group.

Input

```
Edges:
A-B
B-C
E-F
H-I
```

Expected output

```
Number of connected components: 3

Component 1: A B C
Component 2: E F
Component 3: H I
```

Constraints

- Use BFS or DFS.
- Do not use external graph libraries.
- Every node must belong to exactly one component.

Question 6: Detect a Cycle in a Data Pipeline

A data pipeline is represented as a **directed graph**, where an edge $A \rightarrow B$ means that task B depends on task A.

Write a Python program to determine whether the pipeline contains a **cycle**.

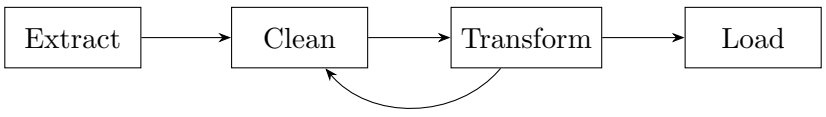
Example 1

```
Extract -> Clean -> Transform -> Load
```

Output:

```
Pipeline is cycle-free
```

Example 2



Output:

```
Pipeline contains a cycle
```

Constraints

- Use DFS.
- Maintain appropriate information about nodes currently being explored.
- Do not use external graph libraries.

Question 7: Data Pipeline Impact Analysis

A data engineering team maintains a pipeline in which tasks depend on one another. The dependencies are given as pairs:

```
raw_customer -> clean_customer
raw_order -> clean_order
clean_customer -> customer_transform
clean_order -> order_transform
customer_transform -> warehouse
order_transform -> warehouse
warehouse -> dashboard
```

Write a Python program that accepts the name of a **failed task** and finds all downstream tasks that may be affected by that failure.

Example

```
Failed task:
customer_transform
```

Expected output

```
Affected tasks:
warehouse
dashboard
```

If the task does not exist:

```
Task not found
```

If the task has no downstream dependencies:

```
No downstream tasks
```

Constraints

- Construct the directed graph from the dependency pairs.
- Use BFS or DFS for traversal.
- Do not hard-code the affected tasks.
- A task must be displayed only once even if it can be reached through multiple paths.
- Handle branching dependencies correctly.

Question 8: Course Scheduling (Topological Sort)

Given n courses, labeled from 0 to $n-1$, and an array `prerequisites[]` where `prerequisites[i] = [x, y]` indicates that we need to take course y first if we want to take course x.

Find an ordering of courses we should take to complete all courses. If there are multiple valid orderings, return any one of them. If it is impossible to finish all courses, return an empty array.

Input

```
n = 4
prerequisites = [[3004, 1023], [3004, 2005], [5006, 3004], [5032, 3004]]
```

Valid outputs

```
[1023, 2005, 3004, 5006, 5032]
[2005, 1023, 3004, 5006, 5032]
[2005, 1023, 3004, 5032, 5006]
```

If it is impossible to complete all courses (i.e., a cycle exists among prerequisites), return:

```
[]
```

Constraints

- Use DFS-based topological sort.
- Maintain appropriate information about nodes currently being explored (to detect cycles).
- Do not use any built-in topological sort or graph library.
- Course labels are not guaranteed to be contiguous from 0 to $n-1$ in the input — build the graph from whatever labels appear in `prerequisites`.

Hint

Use DFS.